

GOOGLE ANTIGRAVITY HACKATHON — INNOCOLLAB.PK

Khidmat

AI Service Orchestrator for Pakistan

Challenge 2 — Informal Economy Platform

CHALLENGE
Challenge 2

TEAM SIZE
4 Members

DEADLINE
1 Week

PLATFORM
Google Antigravity

A complete Product Requirements Document containing the full challenge brief, system architecture, team roles, 7-day execution plan, all Antigravity prompts for all 4 members, API contract, submission checklist, and demo strategy for winning the hackathon.

TABLE OF CONTENTS

SECTION 1	Challenge Brief & Problem Statement	3
SECTION 2	Submission Checklist & Requirements	4
SECTION 3	System Architecture	5
SECTION 4	Tech Stack	6
SECTION 5	Team Roles & Responsibilities	7
SECTION 6	API Contract (Day 1 Agreement)	8
SECTION 7	Day-by-Day Execution Plan	10
SECTION 8	Member 1 — Agent Architect Prompts	13
SECTION 9	Member 2 — Backend Lead Prompts	17
SECTION 10	Member 3 — Mobile App Prompts	21
SECTION 11	Member 4 — Integration Lead Prompts	25
SECTION 12	Stress-Test & Edge Case Scenarios	28
SECTION 13	Submission Checklist & Demo Script	29
SECTION 14	Scoring Strategy	30

Challenge 2: AI Service Orchestrator for Informal Economy

Pakistan's informal economy — plumbers, electricians, tutors, AC technicians, beauticians — operates through WhatsApp, phone calls, and referrals. This causes missed opportunities, poor matching, unpredictable pricing, and no follow-up mechanism. This challenge requires building a full agentic AI system that automates the entire service lifecycle.

What the System MUST Do

- Understand multilingual service requests (Urdu, Roman Urdu, English, code-switched)
- Extract: service type, location, urgency, time preference, constraints, user preferences
- Discover providers using mock data, Google Maps/Places, or other APIs
- Rank providers using multi-factor matching (6+ factors) — not just distance
- Generate dynamic price quotes with transparent breakdown
- Simulate booking, confirmation, reminders, progress, feedback, reputation updates, dispute handling

Example Scenario (from challenge brief)

User Input	"AC bilkul kaam nahi kar raha, kal subah G-13 mein technician chahiye, budget zyada nahi hai."
Understanding	Service=AC repair Severity=High Location=G-13 Time=Tomorrow morning Budget=Sensitive
Matching	Ranks by: travel time, availability, AC specialization, on-time score, review sentiment, rate, cancellation risk
Decision	Recommends Provider A over closer Provider B — higher reliability + specialized AC repair reviews
Pricing	Visit fee + distance cost + urgency adjustment + budget-sensitive alternative shown
Simulation	Books 10AM slot → confirmation → updates booking sheet → reminder → en-route update → feedback → reputa

Evaluation Criteria

Criterion	Weight	What Judges Look For
Antigravity Integration	20%	Core orchestration, reasoning traces, tool calls
Matching & Decision Quality	25%	Multi-factor ranking, explainability, non-trivial decisions
Multilingual Robustness & Edge Cases	15%	Urdu/Roman Urdu/English, misspellings, low confidence
Scheduling, Pricing & Service Workflow	15%	Double booking prevention, dynamic pricing, lifecycle
Dispute Handling, Reliability & Scalability	15%	No-show, cancellation, complaints, fallback
Innovation & UX	10%	Creative approach, mobile UI, compelling demo

SECTION 2 — SUBMISSION CHECKLIST & REQUIREMENTS

What You Must Submit

#	Item	Details
1	Working Prototype	Mobile app is MANDATORY. Web app/dashboard is optional unless needed for demo.
2	Demo Video (main)	3-5 minutes. Must show agentic workflow end to end: request → intent → matching → pricing → booking → follow-up → dispute.
3	Demo Video (Antigravity usage)	2-3 minutes screen recording showing how your team used Antigravity platform specifically.
4	Antigravity Trace/Logs	Workplan, task plan, agent observations, reasoning, decisions, tool calls, action execution, error recovery, final outcomes.
5	README / Documentation	Architecture, data schemas, tools/APIs, Antigravity role, setup steps, assumptions, privacy note, cost/latency, scalability, baseline comparison, limitations.
6	Baseline Comparison	Show how your agentic system performs BETTER than a simple heuristic or non-agentic implementation.
7	Robustness Evidence	Demonstrate at least ONE failure, edge case, contradiction, missing data, or fallback scenario.
8	Cost & Scalability Note	Cost per operation / API call estimate. 10x/100x scaling discussion. Latency or throughput estimate.

■ Critical Reminder

This is NOT a simple booking or listing app. Focus on AGENTIC automation and decision-making.

At least one full booking must be simulated end-to-end with visible system state change.

Must demonstrate reasoning + decision-making at every step (intent → match → price → book → followup).

Antigravity traces MUST be captured and submitted — judges will read them to verify agentic behavior.

High-Level Architecture

The system is structured in three layers: a mobile frontend, a FastAPI backend, and the Antigravity agent orchestration layer. All agent decisions are logged as traces and surfaced back to the mobile app for full transparency.

Layer	Component	Technology	Responsibility
Frontend	Mobile App	React Native (Expo)	User input, results display, booking, agent trace UI
API Layer	REST Backend	FastAPI + Uvicorn	Endpoints, DB access, orchestrator bridge
Database	Local Store	SQLite	Providers, bookings, traces, feedback, disputes
AI Orchestration	Master Orchestrator	Antigravity + Gemini	Runs all 6 agents in sequence
Agent 1	Intent Parser	Gemini 1.5 Flash	Multilingual NLU, confidence scoring
Agent 2	Provider Matcher	Gemini 1.5 Flash	6-factor ranking + reasoning explanation
Agent 3	Pricing Engine	Gemini 1.5 Flash	Dynamic quote: urgency, distance, demand, loyalty
Agent 4	Booking Engine	Gemini 1.5 Flash	Slot confirmation, conflict check, receipt generation
Agent 5	Follow-up Agent	Gemini 1.5 Flash	Reminders, en-route, completion, feedback collection
Agent 6	Dispute Agent	Gemini 1.5 Flash	No-show, complaint, refund, escalation simulation
Logging	Trace Logger	JSON files + SQLite	Full agent trace export for judges

Data Flow: User Request to Confirmed Booking

Step 1	User sends message in Urdu/Roman Urdu/English via mobile chat input
Step 2	POST /orchestrate → FastAPI backend → Antigravity Master Orchestrator
Step 3	Agent 1 (Intent Parser) extracts: service, location, time, urgency, confidence score
Step 4	Agent 2 (Provider Matcher) queries DB, ranks by 6+ factors, generates reasoning text
Step 5	Agent 3 (Pricing Engine) generates dynamic quote with full breakdown
Step 6	Mobile shows results + reasoning + price → user selects provider + confirms
Step 7	Agent 4 (Booking Engine) checks slot conflict → confirms → writes to DB → generates receipt
Step 8	Agent 5 (Follow-up) schedules 3 notifications: 1hr before, 15min before, post-completion
Step 9	Agent 5 collects feedback → Agent 2 updates provider reputation score
Step 10	Full agent trace (6 agents, all decisions) saved to JSON + DB → visible in app

SECTION 4 — TECH STACK

Tool	Category	How It Is Used	Cost
Google Antigravity	AI Orchestration	Core platform — all 6 agents run here. Manager surface for Frontend (chatbot) execution.	FREE
Gemini 1.5 Flash	LLM	Powers all agent reasoning: NLU, matching rationale, pricing logic, dispute handling.	FREE
FastAPI + Uvicorn	Backend	REST API server. All endpoints. Bridge between mobile and agents.	FREE
SQLite	Database	Local database for providers (40+ mock), bookings, traces, feedback for Frontend .	FREE
React Native + Expo	Mobile App	Cross-platform app. Uses React/TypeScript (same as AdVantage App). Runs on Android/iOS.	FREE
Google Cloud Run	Deployment	Deploy FastAPI backend as container. Auto-scales.	FREE tier
Python-dotenv	Config	Manage API keys and environment variables securely.	FREE
Pydantic	Validation	Request/response schema validation in FastAPI.	FREE
httpx	HTTP Client	Async HTTP calls from agents to backend.	FREE

SECTION 5 — TEAM ROLES & RESPONSIBILITIES

Each member works independently on their own GitHub branch and in their own Antigravity workspace. They connect through a shared API contract agreed on Day 1. Member 4 is the integration lead who connects all parts on Day 6.

Member 1	Agent Architect branch: agents	• All 6 Antigravity agents (Intent, Matcher, Pricing, Booking, Follow-up, Dispute) • Master Orchestrator (orchestrator.py) • Gemini API integration and prompt engineering • Agent trace logging and export • Full pipeline tests (test_full_pipeline.py)
Member 2	Backend Lead branch: backend	• FastAPI server + all endpoints • SQLite database (providers, bookings, traces, disputes) • Mock provider dataset (40+ realistic Pakistani providers) • Dynamic pricing logic • API documentation + Postman collection
Member 3	Mobile App Lead branch: mobile	• React Native Expo app (all 7 screens) • API service layer (api.ts) • Agent Trace Card UI (visible reasoning) • Booking receipt and follow-up timeline • App polish, loading states, error handling
Member 4	Integration Lead branch: integration	• Connects all 3 parts end-to-end • Daily integration tests • README documentation • Demo script and video recording • Submission package + trace export

Daily Team Coordination (WhatsApp Standup — 5 min every morning)

Every morning each member sends ONE message in the team group:

Done: [what you completed yesterday]

Today: [what you are building today]

Need: [anything you need from someone else — tag them]

SECTION 6 — API CONTRACT (DAY 1 AGREEMENT)

This document is agreed by all 4 members on Day 1 BEFORE anyone writes code. Once agreed, Member 3 can build the mobile app using fake/mock data, Member 1 can build agents, and Member 2 can build the backend — all independently. On Day 6, Member 4 connects everything using these exact endpoints and JSON shapes.

POST /orchestrate

Main entry point. Runs full agent pipeline.

Request: {"user_message": "...", "user_name": "Ahmad", "user_phone": "0312-XXXXXXX"}

Response: {"intent": {...}, "providers": [...], "pricing": {...}, "agent_trace": [...]}

POST /bookings

Confirms a booking after user selects provider.

Request: {"provider_id": 1, "slot": "10:00 AM", "user_name": "...", "user_phone": "...", "location": "..."}
Response: {"booking_ref": "SVC-2024-XXXX", "status": "confirmed", "receipt": {...}, "followups": [...]}

GET /bookings/{ref}

Get booking details by reference code.

Response: {"booking_ref": "...", "provider": {...}, "status": "...", "slot": "..."}
Response: [{"booking_ref": "...", "service_type": "...", "status": "...}"]

GET /bookings/user/{phone}

Get all bookings for a phone number.

Response: [{"booking_ref": "...", "service_type": "...", "status": "...}"]

DELETE /bookings/{ref}

Cancel a booking.

Response: {"cancelled": true, "message": "Booking cancelled"}

POST /disputes

Report a dispute after service.

Request: {"booking_ref": "...", "type": "no_show|quality|price", "description": "..."}
Response: {"dispute_id": "...", "resolution": "...", "compensation": "..."}
Response: {"steps": [{"agent": "...", "input": "...", "output": "...", "time_ms": 0}]}

GET /traces/{booking_ref}

Get full agent trace for a booking.

Response: {"steps": [{"agent": "...", "input": "...", "output": "...", "time_ms": 0}]}

GET /stats

Dashboard stats for home screen.

Response: {"total_providers": 40, "cities": 4, "avg_rating": 4.5, "bookings_today": 0}

GET /demo/reset

Reset all bookings and traces for demo.

Response: {"cleared": true, "providers_intact": true}

SECTION 7 — 7-DAY EXECUTION PLAN

DAY 1 Setup & API Contract (ALL MEMBERS TOGETHER — 2 hours)	
All	Create GitHub repo, 4 branches, clone on all laptops, open in Antigravity
All	AGREE on API Contract (Section 6) — this is the ONLY thing done together
M1	Set up /agents folder, install dependencies, test first Gemini API call
M2	Set up /backend folder, FastAPI app, SQLite, seed 40 providers
M3	Run npx create-expo-app, set up navigation, build Home screen with mock data
M4	Write master README skeleton, set up GitHub project board
DAY 2 Core Agent + Backend + Mobile Screens	
M1	Build Intent Parser Agent (multilingual NLU, confidence scoring)
M1	Build Provider Matcher Agent (6-factor ranking + Gemini reasoning)
M2	Build GET /providers + POST /orchestrate (skeleton) + provider search
M3	Build Results screen (provider cards + AI reasoning display)
M4	Test M1 and M2 outputs independently, report blockers in standup
DAY 3 Pricing + Booking + Mobile Booking Flow	
M1	Build Pricing Engine Agent (dynamic quote: urgency, distance, demand, loyalty)
M1	Build Booking Engine Agent (slot conflict check, receipt generation)
M2	Build POST /bookings, GET /bookings/{ref}, DELETE /bookings/{ref}
M3	Build Booking screen + Confirmation screen with receipt
M4	Write integration test for full flow (M1 + M2 connected)
DAY 4 Follow-up + Dispute + Remaining Screens	
M1	Build Follow-up Agent (3 notifications + en-route + feedback collection)
M1	Build Dispute Agent (no-show, quality complaint, refund, escalation)
M2	Build POST /disputes, GET /traces/{ref}, GET /stats, GET /demo/reset
M3	Build My Bookings screen + Agent Trace Card (visible reasoning UI)
M4	Run stress-test scenarios, document edge cases found
DAY 5 Integration + Connecting Everything	
M4	Replace all mock data in mobile app with real API calls
M4	Run full end-to-end test: Urdu input → agents → booking → confirmation

M3	Fix any mobile ↔ backend connection issues
M2	Fix any backend ↔ agent issues, ensure CORS works
M1	Ensure all 6 agents produce proper trace logs saved to JSON

DAY 6 Polish + Demo Prep	
All	Run all 5 stress-test scenarios and fix failures
M3	Polish UI: loading states, error states, animations
M1	Make agent reasoning VISIBLE in app (this is 45% of your score)
M4	Capture all Antigravity trace logs for submission
M4	Write final README with architecture, cost analysis, baseline comparison

DAY 7 Demo Video + Submission	
All	Record 3-5 minute demo video following the demo script (Section 13)
All	Record 2-3 minute screen recording of Antigravity usage
M4	Final README review, export traces, zip submission package
All	Submit on innocollab.pk before deadline

SECTION 8 — MEMBER 1: AGENT ARCHITECT PROMPTS

DAY 1 — Project Setup + Gemini Client

Prompt 1.1 — Project Structure

Create a Python project at /agents with this structure:
/intent_parser/agent.py, /provider_matcher/agent.py,
/pricing_engine/agent.py, /booking_engine/agent.py,
/followup_agent/agent.py, /dispute_agent/agent.py,
/shared/gemini_client.py, /shared/models.py,
orchestrator.py, requirements.txt, .env.example
Dependencies: google-generativeai, python-dotenv, pydantic, httpx, pytest

Prompt 1.2 — Gemini Client

In /agents/shared/gemini_client.py build a Gemini API client:
- Load GEMINI_API_KEY from .env
- Model: gemini-1.5-flash
- call_gemini(system_prompt, user_message) -> str with 3-retry logic
- call_gemini_json(system_prompt, user_message) -> dict, strips markdown
- Log every call with timestamp and response time
- Handle quota exceeded and network errors gracefully

DAY 2 — Intent Parser + Provider Matcher Agents

Prompt 2.1 — Intent Parser Agent

Build IntentParserAgent in /agents/intent_parser/agent.py
Method: parse(user_message: str) -> dict

System prompt for Gemini (save in prompts.py):
"You are an intent parser for a Pakistani home services app.
Extract these fields from the user message:
- service_type: AC Technician, Plumber, Electrician, Tutor, Beautician, Carpenter
- location: area or city mentioned (G-13, DHA, Gulshan, etc.)
- time_preference: when they want service
- urgency: urgent/normal
- budget_sensitivity: high/normal/low
- job_complexity: basic/intermediate/complex (estimate from description)
- language_detected: urdu/roman_urdu/english/mixed
- confidence_score: 0.0 to 1.0
- confirmation_needed: true if confidence < 0.7
Return ONLY valid JSON. If field not mentioned set to null."
Return parsed dict + add field parsing_confidence: high/medium/low

Prompt 2.2 — Provider Matcher Agent

Build ProviderMatcherAgent in /agents/provider_matcher/agent.py
Method: match(parsed_intent: dict, providers_list: list) -> dict

Use Gemini to rank providers by ALL 6 factors:
1. Distance/travel time (closer = better)
2. Availability (available slots matching time_preference)
3. Rating + review recency (recent 5-star beats old 5-star)
4. Reliability/on-time score
5. Skill specialization match to job_complexity
6. Price vs budget_sensitivity
7. Cancellation rate (lower = better)

System prompt must ask Gemini to:
- Return ranked list with match_score (0-100)
- Give ONE reasoning sentence per provider in user language

- Flag if best pick is NOT the closest (explain why)
Return: ranked_providers, overall_recommendation, decision_explanation

DAY 3 — Pricing Engine + Booking Engine Agents

Prompt 3.1 — Pricing Engine Agent

Build PricingEngineAgent in /agents/pricing_engine/agent.py
Method: generate_quote(provider, parsed_intent) -> dict

Price calculation factors:

- base_rate: provider.price_per_hour
- distance_cost: PKR 50 per km over 3km
- urgency_surcharge: +20% if urgency=urgent
- complexity_multiplier: 1.0 basic, 1.3 intermediate, 1.6 complex
- demand_factor: +10% if all similar providers busy
- loyalty_discount: -5% if user has previous bookings
- budget_alternative: if budget_sensitivity=high, show cheaper option

Return: {
quoted_price, price_breakdown, budget_alternative,
fairness_note, transparency_message
}

Prompt 3.2 — Booking Engine Agent

Build BookingEngineAgent in /agents/booking_engine/agent.py
Method: book(provider_id, slot, user_name, user_phone, service_type, location) -> dict

Steps:

1. Call GET /providers/{id} to verify availability
2. Check slot not already booked (call backend)
3. If conflict: suggest next 2 available slots
4. If OK: call POST /bookings on backend
5. Use Gemini to generate Hinglish confirmation message
6. Return full receipt

Method: cancel(booking_ref) -> dict

Method: reschedule(booking_ref, new_slot) -> dict (if provider cancels)

Handle: slot taken, provider unavailable, network error

DAY 4 — Follow-up + Dispute Agents + Master Orchestrator

Prompt 4.1 — Follow-up Agent

Build FollowupAgent in /agents/followup_agent/agent.py
Method: schedule_followups(booking: dict) -> dict

Create 5 simulated notifications:

1. Confirmation: immediate – Hinglish booking confirmed message
2. Reminder: 1 hour before – warm reminder to be home
3. En-route: 15 min before – "Provider is on the way!"
4. Completion: after service – request feedback
5. Reputation update: after feedback – update provider score

Method: collect_feedback(booking_ref, rating, comment) -> dict

- Updates provider reputation score in DB
- Returns updated provider profile

All notifications logged to /agents/followup_logs/{booking_ref}.json

Prompt 4.2 — Dispute Agent

```
Build DisputeAgent in /agents/dispute_agent/agent.py
Method: handle_dispute(booking_ref, dispute_type, description) -> dict
```

dispute_type options:

- no_show: provider never arrived
- quality_complaint: poor work quality
- price_disagreement: charged more than quoted
- overrun: took much longer than estimated

For each type Gemini generates:

- Resolution recommendation
- Compensation amount (if applicable)
- Escalation decision (auto-resolve vs human escalation)
- Provider flag (warning/blacklist if pattern detected)

Method: escalate_to_human(dispute_id) -> dict

Return: {dispute_id, resolution, compensation, provider_action, message}

Prompt 4.3 — Master Orchestrator

```
Build ServiceOrchestrator in /agents/orchestrator.py
```

Method: process_request(user_message, user_name, user_phone) -> dict

PIPELINE (log each step):

- Step 1: Intent Parser – parse message, get confidence
- Step 2: If confidence < 0.7, return clarification_needed=True with question
- Step 3: Provider Discovery – GET /providers?service=X&location=Y
- Step 4: Provider Matcher – rank with reasoning
- Step 5: Pricing Engine – generate quotes for top 3 providers
- Step 6: Return to user: intent + ranked_providers + prices + agent_trace

Method: confirm_booking(provider_id, slot, user_name, user_phone, ...) -> dict

- Step 7: Booking Engine – confirm slot, check conflicts
- Step 8: Follow-up Agent – schedule all 5 notifications
- Step 9: Save full trace (all 6 agents) to JSON + POST /traces
- Step 10: Return booking_receipt + followup_schedule + agent_trace

CRITICAL: agent_trace must have ALL steps with timing.

Save traces to /agents/traces/{booking_ref}_{timestamp}.json

DAY 5-6 — Stress Tests + Edge Cases

Prompt 5.1 — Stress Test Scenarios

Create /agents/test_stress.py with 5 stress-test scenarios:

Scenario 1 – No providers available:

Input: "Mujhe abhi plumber chahiye G-100 mein" (fake area)

Expected: helpful message asking for different area/time

Scenario 2 – Provider cancels after booking:

Simulate: book slot → mark provider unavailable → call reschedule()

Expected: system finds next available slot automatically

Scenario 3 – Ambiguous/misspelled input:

Input: "AC wala bhejo jldi F-10 mein" (misspelling + urgency)

Expected: confidence_score < 0.7, confirmation question sent

Scenario 4 – Double booking conflict:

Two users book same provider same slot simultaneously

Expected: second user gets "slot taken" + next available slot

Scenario 5 – Post-service dispute:

Input: quality_complaint after completed service
Expected: compensation offered, provider flagged

Print PASS/FAIL for each scenario. These become your robustness evidence for submission.

SECTION 9 — MEMBER 2: BACKEND LEAD PROMPTS

DAY 1 — Project Setup + Database

Prompt 1.1 — FastAPI Project + Database

Create /backend with: main.py, database.py, models.py, schemas.py, routers/providers.py, routers/bookings.py, routers/disputes.py, routers/agents.py, data/seed_providers.py, requirements.txt

SQLAlchemy + SQLite tables:

providers: id, name, owner_name, service_type, area, city, rating, total_reviews, on_time_score, cancellation_rate, experience_years, skill_level (basic/intermediate/complex), price_per_hour (PKR), distance_km, available, phone, slots (JSON string), created_at

bookings: id, booking_ref (SVC-YYYY-XXXX), provider_id (FK), user_name, user_phone, service_type, location, slot, status, quoted_price, actual_price, notes, created_at, confirmed_at

disputes: id, booking_ref, dispute_type, description, status, resolution, compensation, provider_flag, created_at

agent_traces: id, booking_ref, trace_data (JSON text), created_at

Prompt 1.2 — Seed 40 Providers

Seed 40 realistic Pakistani providers across 4 cities:

10 Islamabad (G-13, F-10, I-8, G-9, F-7)
10 Karachi (DHA, Gulshan, Defence, Clifton, PECHS)
10 Lahore (DHA, Gulberg, Model Town, Johar Town, Bahria)
10 Rawalpindi (Saddar, Bahria, PWD, Chaklala, Westridge)

Services: AC Technician (8), Plumber (8), Electrician (8), Tutor (6), Beautician (4), Carpenter (3), Painter (3)

Use realistic Pakistani names. Rating: 3.8-4.9. Price:
AC Tech 800-1500 PKR/hr, Plumber 500-1200, Electrician 600-1300
on_time_score: 0.7-1.0. cancellation_rate: 0.0-0.3
skill_level: 40% basic, 40% intermediate, 20% complex
80% available=True. 3-4 time slots per provider.

DAY 2-3 — API Endpoints

Prompt 2.1 — Providers Router

In /backend/routers/providers.py:

GET /providers – list all, filter by service_type, city, area, available_only
GET /providers/{id} – single provider
GET /providers/search?service_type=X&location=Y&available_only=true
Smart location: "G-13" matches area, "Islamabad" matches city
Returns: providers sorted by available DESC, rating DESC, distance ASC
Add fields: distance_label "2.1 km away", price_label "Rs. 800-1,200/hr"
GET /providers/services/list – all unique service types

Prompt 2.2 — Bookings + Disputes + Traces

POST /bookings – create booking, check slot conflict, generate SVC-ref
GET /bookings/{ref} – booking + provider details
GET /bookings/user/{phone} – all user bookings sorted by date
DELETE /bookings/{ref} – cancel (status=cancelled)
POST /bookings/{ref}/complete – mark completed

```
POST /disputes – create dispute record
GET /disputes/{ref} – get dispute + resolution

POST /traces – save agent trace JSON
GET /traces/{booking_ref} – retrieve trace
GET /traces/latest – last 5 traces (for demo dashboard)

GET /health – {status, db: connected, providers_count, bookings_count}
GET /stats – {total_providers, available_now, cities, avg_rating, bookings_today}
GET /demo/reset – clear bookings+traces, keep providers
```

DAY 4 — Dynamic Pricing Logic

Prompt 3.1 — Pricing Endpoint

Add POST /pricing/quote to backend:

Body: {provider_id, service_type, location, urgency, job_complexity, user_phone}

Calculate price:

```
base = provider.price_per_hour
+ distance_cost = max(0, distance_km - 3) * 50 PKR
+ urgency_surcharge = base * 0.20 if urgency=urgent
* complexity_mult: 1.0 basic, 1.3 intermediate, 1.6 complex
+ demand_surcharge = base * 0.10 if all providers of type busy
- loyalty_discount = 0.05 if user has 3+ previous bookings
```

Return: {quoted_price, breakdown as dict, budget_alternative, min_price (if budget_alternative), fairness_score 0-100}

Also add input validation for all POST endpoints:

Pakistani phone format: 03XX-XXXXXXX or 03XXXXXXXXXX

Slot format: HH:MM AM/PM. provider_id must exist.

SECTION 10 — MEMBER 3: MOBILE APP PROMPTS

DAY 1 — Expo Setup + Navigation + Home Screen

Prompt 1.1 — Project Setup

```
Run: npx create-expo-app KhidmatApp --template blank-typescript

Install:
@react-navigation/native @react-navigation/stack
react-native-safe-area-context react-native-screens
expo-linear-gradient @expo/vector-icons expo-haptics

Create structure:
/src/screens: Home, Results, Booking, Confirmation, MyBookings, AgentTrace
/src/components: ProviderCard, AgentTraceCard, PricingCard,
LoadingDots, BookingReceipt, NotificationTimeline
/src/services/api.ts
/src/types/index.ts
/src/constants/colors.ts (primary: #1B5E20, accent: #4CAF50, gold: #F9A825)
```

Prompt 1.2 — Home Screen

Build HomeScreen.tsx:

1. Header: App name "Khidmat" (■■■■■) + "Pakistan's AI Service Finder"
2. Stats bar: providers count, cities, avg rating (from GET /stats)
3. Service quick-select buttons (horizontal scroll):
AC Tech *, Plumber ■, Electrician ■, Tutor ■, Beautician ■
4. Large text input (multiline):
Placeholder: "Apni zaroorat batayein... (Urdu, Roman Urdu, ya English)"
5. Example query chips: "Mujhe plumber chahiye", "AC repair G-13"
6. On send: validate not empty → navigate to Results with message param
Use keyboard avoiding view. Haptic feedback on send button.

DAY 2-3 — Results + Booking + Confirmation Screens

Prompt 2.1 — Provider Card Component

Build ProviderCard.tsx showing:

- Rank badge (#1 gold, #2 silver, #3 bronze)
- Provider name + owner name
- Service type (green pill) + area + distance
- Rating stars + on_time_score + experience
- Price range in PKR
- Available slots (highlight recommended slot)
- AI Reasoning box (green background, robot emoji, reasoning text)
- Booking button (disabled if unavailable)
- "Why this provider?" expandable section showing full decision trace

Prompt 2.2 — Results Screen

On mount: call POST /orchestrate with userMessage.
Show loading animation cycling messages:
"Understanding your request...", "Searching providers...",
"Ranking best matches...", "Calculating prices..."

After loading show:

1. Intent card: what AI understood (service, location, time, language, confidence)
2. Provider cards (FlatList, ranked)
3. Pricing cards per provider
4. Agent Trace section (collapsible) showing 6 steps

Handle: error state with retry, empty state (no providers found)

Prompt 2.3 — Booking + Confirmation Screens

BookingScreen: provider summary + form (name, phone, address, notes, slot)
Validate Pakistani phone format. Show price breakdown. Confirm button.

ConfirmationScreen:

- Success animation (green checkmark)
- "Booking Ho Gayi!" header
- Full receipt card (booking_ref, provider, slot, price, status)
- Notification timeline (5 notifications with times)
- AI confirmation message (Hinglish, from Gemini)
- "View My Bookings" + "Book Another Service" buttons

DAY 4 — Agent Trace UI + My Bookings + Polish

Prompt 3.1 — Agent Trace Card (CRITICAL FOR JUDGES)

Build AgentTraceCard.tsx – this is the most important component for judges.

Show 6 agent steps as a visual timeline:

- Step 1: Intent Parser – input message → extracted intent
- Step 2: Provider Discovery – query → N providers found
- Step 3: Provider Matcher – ranking factors → top pick + score
- Step 4: Pricing Engine – calculation → quoted price
- Step 5: Booking Engine – slot check → confirmation
- Step 6: Follow-up Scheduler – N notifications scheduled

Each step shows: agent name, input summary, output summary, time_ms

Color: completed=green, active=yellow pulse, pending=gray

"Export Trace" button → copies JSON to clipboard

Title: "AI Agent Reasoning Trace – 6 steps"

This proves agentic workflow to judges.

Prompt 3.2 — My Bookings + Dispute Screen

MyBookingsScreen: phone input → GET /bookings/user/{phone}

Show each booking as card with status badge.

Confirmed: Cancel button. Completed: Rate + Report Issue buttons.

Cancelled: Rebook button.

DisputeScreen (reach from completed booking):

Select dispute type (no_show/quality/price). Text description.

Submit → POST /disputes → show resolution + compensation message.

This is your robustness evidence for judges.

SECTION 11 — MEMBER 4: INTEGRATION LEAD PROMPTS

DAY 1 — Documentation + Setup

Prompt 1.1 — README Structure

```
Create /README.md with sections:
# Khidmat — AI Service Orchestrator for Pakistan
## Problem, Solution, Architecture Diagram (ASCII)
## How Google Antigravity is Used (each of 6 agents)
## Agent Workflow (step-by-step with timing)
## Tech Stack table
## API Endpoints table
## Provider Dataset Schema (fields + sample)
## Matching Algorithm (6 factors explained)
## Dynamic Pricing Formula
## Cost Analysis (per API call, 10x/100x scaling)
## Baseline Comparison (agentic vs non-agentic)
## Privacy Note (no real personal data)
## Assumptions + Limitations
## Setup Instructions (backend + agents + mobile)
## Team Members + Roles
```

Prompt 1.2 — Environment + Scripts

```
Create /scripts/setup.sh, /scripts/start.sh, /scripts/reset_demo.sh
setup.sh: install deps for backend + agents, create .env from .env.example
start.sh: start backend on :8000, agents on :8001, print health check URL
reset_demo.sh: call GET /demo/reset, print "Ready for judges"
Create docker-compose.yml for backend + agents together.
```

DAY 2-4 — Integration Testing

Prompt 2.1 — End-to-End Integration Tests

```
Create /integration_tests/test_e2e.py

Session 1 — Roman Urdu, Islamabad:
POST /orchestrate: "Mujhe kal subah G-13 mein AC technician chahiye"
Assert: intent.service_type="AC Technician", providers found > 0, trace has 6 steps
Then POST /bookings with top provider. Assert booking_ref starts with SVC-.
Then GET /traces/{ref}. Assert 6 steps all completed.

Session 2 — English, Karachi:
"Need a plumber in DHA Karachi today evening"
Full flow same as Session 1.

Session 3 — Edge case (no location):
"mujhe electrician chahiye" → assert clarification_needed=True

Session 4 — Dispute flow:
Complete booking → POST /disputes (quality_complaint) → assert resolution exists

Print: PASS/FAIL for each. Save to /integration_tests/results.json
```

Prompt 2.2 — Baseline Comparison Document

```
Create /demo/baseline_comparison.md

Compare your agentic system vs a simple non-agentic approach:

Non-agentic approach: user types service → show all providers by distance → user picks → done
Your system: 6-agent pipeline with NLU, 6-factor ranking, dynamic pricing, booking conflict check,
```

follow-up, dispute handling

Comparison table:

Feature	Simple App	Khidmat (Agentic)
Language Support	English only	Urdu + Roman Urdu + English
Ranking factors	Distance only	6+ factors with AI reasoning
Pricing	Fixed rate	Dynamic (urgency, demand, loyalty)
Double booking	Not prevented	Agent checks conflicts
Follow-up	None	5 automated notifications
Disputes	Not handled	Full resolution workflow
Transparency	None	Full agent trace visible

This is a REQUIRED submission item.

DAY 5-6 — Demo Preparation

Prompt 3.1 — Demo Script

Create /demo/demo_script.md with full 5-minute recording script:

[0:00-0:30] INTRO: App name, the Pakistan informal economy problem
[0:30-1:00] LIVE INPUT: Type in Roman Urdu – show AI understanding it
[1:00-2:00] RESULTS: Show ranked providers + AI reasoning card + prices
[2:00-2:30] AGENT TRACE: Expand trace – walk through all 6 agent steps
[2:30-3:30] BOOKING: Fill form → confirmation screen → receipt → notifications
[3:30-4:00] EDGE CASE: Show no-provider scenario (robustness evidence)
[4:00-4:30] DISPUTE: Show dispute flow + resolution
[4:30-5:00] CLOSING: Architecture, Antigravity role, impact statement

Also create /demo/demo_inputs.txt with exact text to type during recording.

Create /demo/final_checklist.md with every submission requirement checked.

Prompt 3.2 — Trace Viewer Dashboard

Create /demo/trace_viewer.html – single HTML file, no dependencies.

Dark theme. Auto-refresh every 5 seconds from GET /traces/latest.

Show last 5 traces as cards with:

- Booking ref + timestamp + LIVE badge if < 2 minutes old
- User message that triggered it
- 6-step pipeline visualization (colored boxes + arrows)
- Time per step in ms
- Final result summary

Use fetch() to call http://localhost:8000/traces/latest.

Show on second screen during demo to impress judges.

SECTION 12 — STRESS-TEST & EDGE CASE SCENARIOS

These 5 scenarios are REQUIRED by the submission checklist as robustness evidence. Member 1 builds them into test_stress.py. Member 4 runs them and documents results. Show at least ONE of these in your demo video.

Scenario 1: No providers available	Input	"Mujhe abhi plumber chahiye G-100 mein" (non-existent area)
	Expected	System returns: no_providers_found=True, helpful_message asking for different area or time, suggests nearest available area
Scenario 2: Provider cancels after confirmation	Input	Book a slot → mark provider unavailable in DB → trigger reschedule()
	Expected	System auto-finds next available slot, sends updated confirmation, logs the rescheduling in trace
Scenario 3: Ambiguous/misspelled input	Input	"AC wala bhejo jldi F-10 mein" (misspelling + urgency + mixed language)
	Expected	confidence_score < 0.7, confirmation_needed=True, system asks clarifying question before proceeding
Scenario 4: Simultaneous double booking	Input	Two users book same provider same slot at the same time
	Expected	Second user gets slot_conflict error + next 2 available slots suggested automatically
Scenario 5: Price dispute after service	Input	User reports charged more than quoted price
	Expected	Dispute agent calculates compensation, flags provider, sends resolution message, updates provider record

SECTION 13 — SUBMISSION CHECKLIST & DEMO SCRIPT

Pre-Submission Checklist

Functionality

- Home screen loads with live stats
- Roman Urdu input works end-to-end
- English input works end-to-end
- Provider results appear with AI reasoning
- Booking form validates Pakistani phone
- Confirmation screen shows receipt + notifications
- Agent trace shows all 6 steps
- Dispute flow works and shows resolution
- Edge case: no providers → helpful message shown

Antigravity Requirements

- All 6 agents built and running in Antigravity
- Agent reasoning visible in mobile app UI
- Traces saved and retrievable via API
- Workplan and task plan artifacts captured
- Error recovery scenario demonstrated

Submission Items

- Mobile app tested on actual phone (not just emulator)
- Demo video: 3-5 min (main) + 2-3 min (Antigravity usage)
- Antigravity trace/logs exported as JSON
- README complete with all required sections
- Baseline comparison document included
- Robustness evidence documented
- Cost per API call estimated in README
- GitHub repo public and clean

5-Minute Demo Script

Time	Section	Script / What to Show
[0:00 - 0:30]	INTRO	Show home screen. "Pakistan mein millions of people need home services daily. Khidmat automates this with 6 AI agents."
[0:30 - 1:30]	LIVE ROMAN URDU INPUT	Type: "AC bilkul kaam nahi kar raha, kal subah G-13 mein technician chahiye, budget zyada nahi hai." Show AI parsing animation.
[1:30 - 2:30]	RESULTS + AI REASONING	Show 3 ranked providers. Open one AI Reasoning card. Explain: "AI chose Provider A over closer Provider B because of higher reliability score."

[2:30 - 3:00]	AGENT TRACE	Expand Agent Trace Card. Walk through all 6 steps. Show timing. "This is full transparency — you can see every agent decision."
[3:00 - 3:45]	BOOKING + CONFIRMATION	Fill booking form → confirm → show receipt with booking ref → show 5 notification timeline.
[3:45 - 4:15]	ROBUSTNESS (EDGE CASE)	Type: "mujhe koi bhi chahiye" (vague). Show confidence_score < 0.7 → clarification question. This is your robustness evidence.
[4:15 - 4:45]	DISPUTE HANDLING	Go to My Bookings → Report Issue → quality complaint → show resolution + compensation. "Even after service, the system handles disputes."
[4:45 - 5:00]	CLOSING	Show architecture diagram. "Built on Google Antigravity with 6 specialized agents. Thank you."

SECTION 14 — SCORING STRATEGY

How to maximize your score on every criterion:

Antigravity Integration	20%
✓ Build 6 distinct agents — each does ONE specific job	
✓ Capture and save ALL agent traces with workplan + task plan artifacts	
✓ Show Antigravity Manager surface in your 2-3 min screen recording	
✓ Demonstrate error recovery: show an agent handling a failure	
Matching & Decision Quality	25%
✓ Use ALL 6 factors (not just distance) — name them in demo	
✓ Show reasoning text per provider: "Chose A over B because..."	
✓ Show a case where closest provider is NOT the top pick (explain why)	
✓ Confidence score visible when input is ambiguous	
Multilingual Robustness	15%
✓ Show Urdu, Roman Urdu, and English inputs in demo	
✓ Demonstrate misspelling handling (jldi, F-10, etc.)	
✓ Show low-confidence case triggering clarification question	
✓ Show code-switched input: "Mujhe kal morning main AC service chahiye"	
Scheduling, Pricing & Workflow	15%
✓ Show complete booking lifecycle in demo (all 7 steps)	
✓ Show dynamic pricing breakdown (base + surcharges + discounts)	
✓ Show double booking prevention in stress test	
✓ Show 5 follow-up notifications timeline in UI	
Dispute, Reliability & Scalability	15%
✓ Show dispute flow in demo (quality complaint scenario)	
✓ Include cost/latency estimates in README	
✓ Include 10x/100x scaling discussion in README	
✓ Document baseline comparison (agentic vs simple app)	
Innovation & UX	10%
✓ Urdu text support in app ("Khidmat" branding)	

✓ Agent trace UI visible inside the app — not just in logs

✓ Clean mobile UI with green Pakistan theme

✓ Trace viewer dashboard on second screen during demo

THE SINGLE BIGGEST TIP

Show the agent reasoning VISIBLY inside the mobile app UI — not just in logs or terminal.

The AgentTraceCard component (Section 10, Prompt 3.1) covers 45% of your total score.

Judges want to SEE the AI thinking, not just trust that it happened.

Every decision (why Provider A, why this price, why this slot) must be readable in the app.